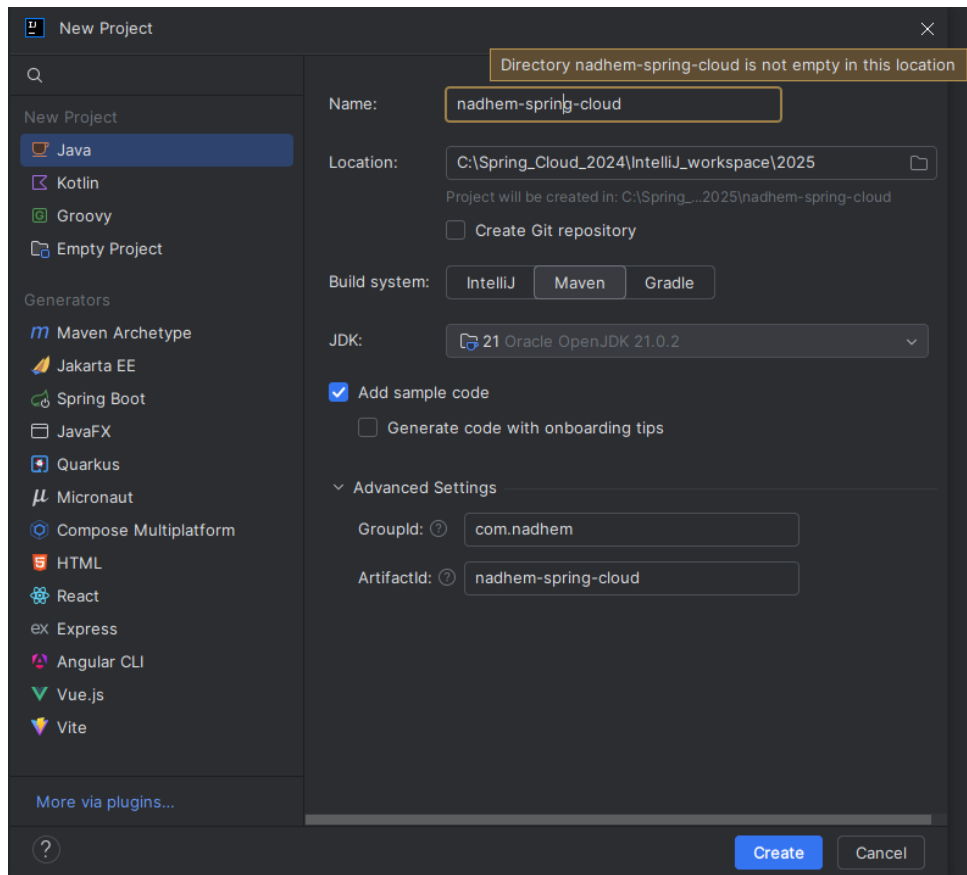


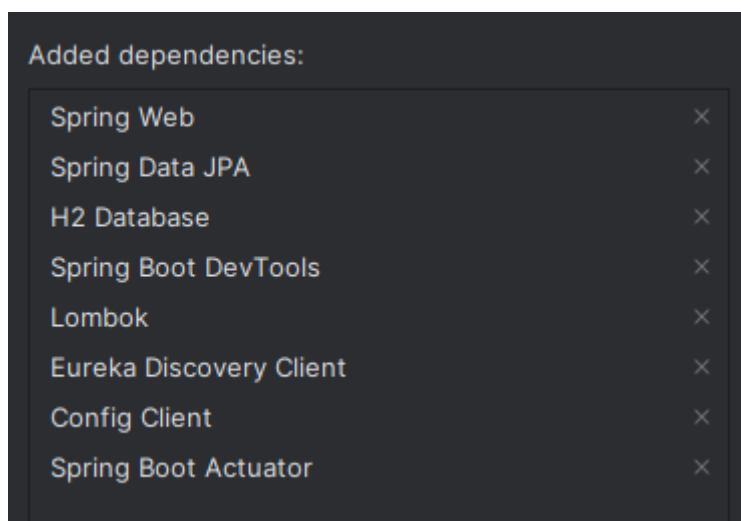
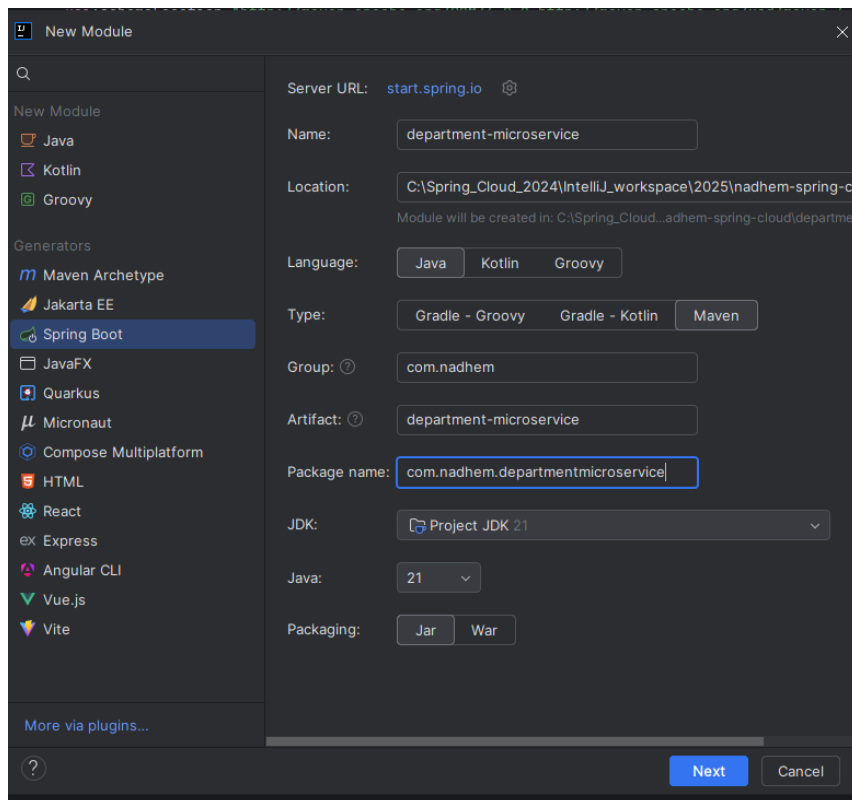
Création des microservices

Objectifs :

1. Créer le micro service department-microservice
2. Créer le micro service teacher-microservice

Créer le micro service department-microservice





Configuration du fichier application.yml

1. Modifier le fichier src/main/resources/application.properties comme suit :

```
spring:
  application:
    name: department
  datasource:
    url: 'jdbc:h2:mem:depdb'
  h2:
    console:
      enabled: true
  jpa:
    show-sql: true
    hibernate:
      ddl-auto: update
  cloud:
    config:
      enabled: false
    discovery:
      enabled: false
  management:
    endpoints:
      web:
        exposure:
          include: '*'
```

Création de l'entité Department et de son interface Repository

```
package com.department.entities;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String depName;
    private String depCode;
}
```

```
package com.nadhemb.department.repos;

import com.nadhemb.department.entities.Department;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface DepartmentRepository extends JpaRepository<Department,
Long> {
}
```

Créer la classe DepartmentDto

```
package com.nadhem.department.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class DepartmentDto {
    private Long id;
    private String DepName;
    private String DepCode;
}
```

Ajouter la méthode `findByDepCode` à l'interface `DepartmentRepository`

```
Department findByDepCode(String code);
```

Création du service

L'interface `DepartmentService`

```
package com.nadhem.department.service;

import com.nadhem.department.dto.DepartmentDto;

public interface DepartmentService {
    DepartmentDto getDepartmentByCode(String code);
}
```

La classe `DepartmentServiceImpl`

```
package com.nadhem.department.service;

import com.nadhem.department.dto.DepartmentDto;
import com.nadhem.department.entities.Department;
import com.nadhem.department.repos.DepartmentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class DepartmentServiceImpl implements DepartmentService {
    @Autowired
    DepartmentRepository departmentRepository;

    @Override
    public DepartmentDto getDepartmentByCode(String code) {
        Department dep = departmentRepository.findByDepCode(code);
        DepartmentDto departmentDto = new DepartmentDto(
            dep.getId(),
            dep.getDepName(),
            dep.getDepCode()
        );
    }
}
```

```

        return departmentDto;
    }
}

```

Création de la classe DepartmentController

```

package com.nadhem.department.restControllers;

import com.nadhem.department.dto.DepartmentDto;
import com.nadhem.department.service.DepartmentService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import lombok.AllArgsConstructor;

@RestController
@RequestMapping("/api/departments")
@AllArgsConstructor
public class DepartmentController {
    private DepartmentService departmentService;

    @GetMapping("/{code}")
    public ResponseEntity<DepartmentDto> getDepByCode(@PathVariable("code")
String code )
    {
        return new ResponseEntity<DepartmentDto>(
            departmentService.getDepartmentByCode(code),
            HttpStatus.OK);
    }
}

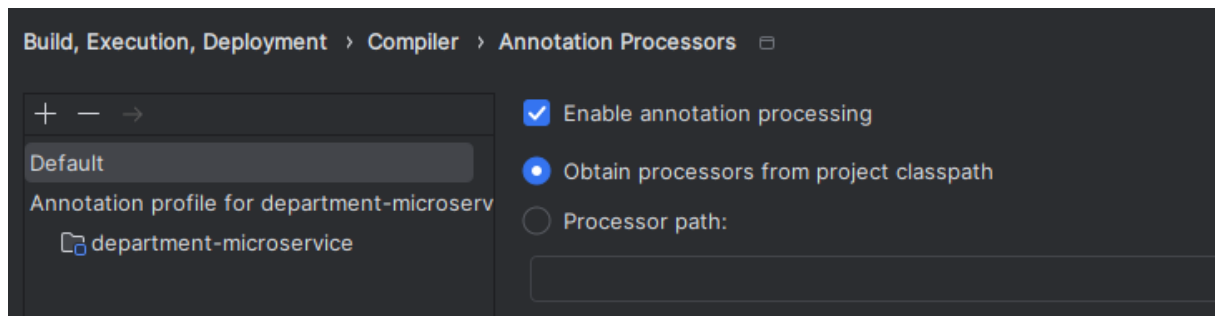
```

Ajouter 2 départements au démarrage :

```

@Bean
CommandLineRunner commandLineRunner(DepartmentRepository departmentRepository) {
    return args -> {
        departmentRepository.save(Department.builder()
            .depName("Info Tech")
            .depCode("IT")
            .build());
        departmentRepository.save(Department.builder()
            .depName("Marketing")
            .depCode("MK")
            .build());
    };
}

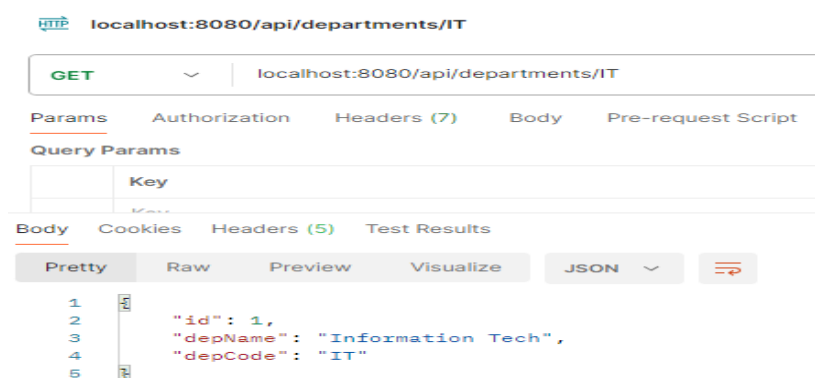
```



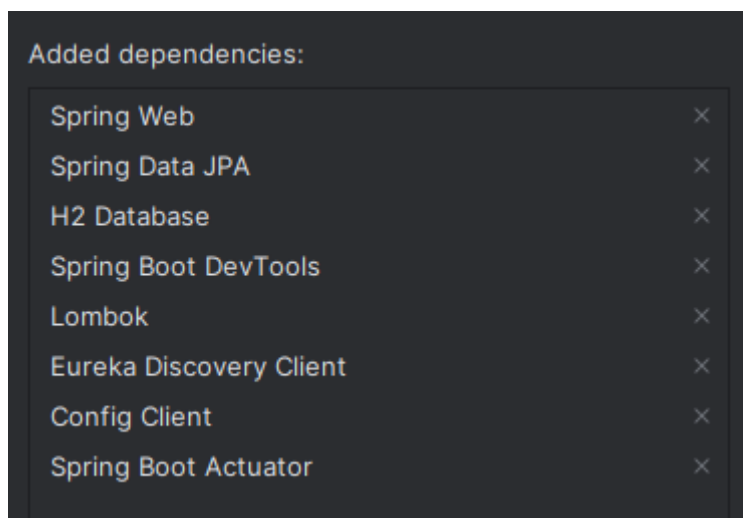
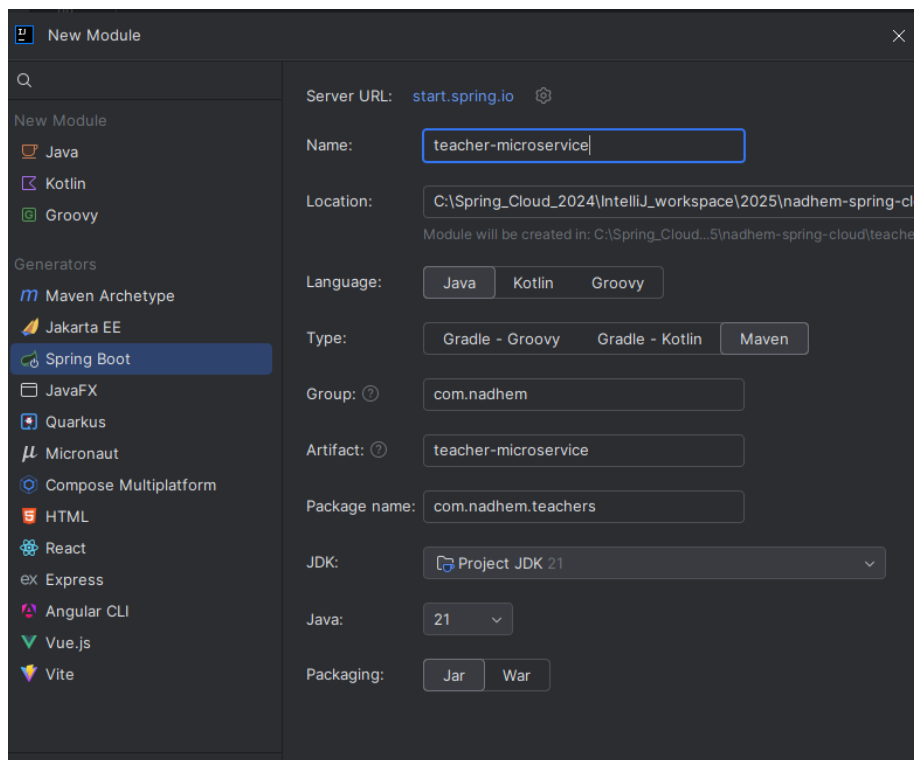
Si vous avez des problèmes avec lombok (dernières versions de IntelliJ), Au niveau du fichier pom.xml, remplacez la balise Build par :

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>${lombok.version}</version>
          </path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Testez avec Postman votre microservice :



Créer le microservice teacher-microservice



Configuration du fichier application.yml

```
server:
  port: 8081
spring:
  application:
    name: teacher
  datasource:
    url: 'jdbc:h2:mem:teachersdb'
  h2:
    console:
      enabled: true
  jpa:
    show-sql: true
    hibernate:
      ddl-auto: update
  cloud:
    config:
      enabled: false
    discovery:
      enabled: false
  management:
    endpoints:
      web:
        exposure:
          include: '*'
```

Création de l'entité Teacher et de son interface Repository

```
package com.nadhem.teachers.entities;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
public class Teacher {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
}
```

L'interface TeacherRepository


```
package com.nadhem.teachers.repos;

import org.springframework.data.jpa.repository.JpaRepository;
import com.nadhem.teacher.entities.Teacher;

public interface TeacherRepository extends JpaRepository<Teacher, Long> {

}
```

Créer la classe TeacherDto

```
package com.nadhem.teachers.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class TeacherDto {

    private Long id;
    private String firstName;
    private String lastName;

}
```

Création du service

```
package com.nadhem.teachers.service;

import com.nadhem.teacher.dto.TeacherDto;

public interface TeacherService {

    TeacherDto getTeacherById(Long id);

}
```

La classe TeacherServiceImpl

```
package com.nadhem.teachers.service;

import com.nadhem.teacher.dto.TeacherDto;
import com.nadhem.teacher.entities.Teacher;
import com.nadhem.teacher.repos.TeacherRepository;
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Service;

@AllArgsConstructor
@Service
public class TeacherServiceImpl implements TeacherService {

    private TeacherRepository teacherRepository;

    @Override
    public TeacherDto getTeacherById(Long id) {
        Teacher teacher = teacherRepository.findById(id).get();
        return new TeacherDto(
            teacher.getId(),

```

```

        teacher.getFirstName(),
        teacher.getLastName()
    );
}
}

```

Création de la classe TeacherController

```

package com.nadhem.teacher.restControllers;

import com.nadhem.teachers.dto.TeacherDto;
import com.nadhem.teachers.service.TeacherService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import lombok.AllArgsConstructor;

@RestController
@RequestMapping("/api/teachers")
@AllArgsConstructor
public class TeacherController {

    private TeacherService teachersService;

    @GetMapping("/{id}")
    public ResponseEntity<TeacherDto> getTeacherById(@PathVariable("id")
Long id )
    {
        return new ResponseEntity<TeacherDto>(
            teachersService.getTeacherById(id), HttpStatus.OK);
    }
}

```

Ajouter un enseignant au démarrage :

```

@Bean
CommandLineRunner commandLineRunner(TeacherRepository teacherRepository) {
    return args -> {
        teacherRepository.save(Teacher.builder()
            .firstName("Nadhem")
            .lastName("Bel Hadj")
            .build());
    };
}

```

Si vous avez des problèmes avec lombok (dernières versions de IntelliJ), Au niveau du fichier pom.xml, remplacez la balise Build par :

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>${lombok.version}</version>
          </path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Testez avec Postman votre microservice :

